République algérienne démocratique et populaire Ministère de l'enseignement supérieur et de la recherche scientifique universitaire de Tahri Mohamed Béchar
Faculté des Sciences Exactes
Département Mathématiques et informatique

Système d'Information

Base de données Orienté Objet
(objet – relationnel)

Dr. Djelloul Kheira

Etudiants concernés

Faculté/Institut	Département	Niveau	Spécialité
Science Exactes	MI	ING3	Informatique

Limite de SGBD Relationnel

Force du Modèle Relationnel

- Le modèle de données est très simple (tables, relations…) et donc facile à comprendre pour les utilisateurs.
- Le modèle repose sur une base formellement définie (normalisation, algèbre, …), ce qui a permis de définir des langages de manipulation de données (LMD) standardisés tels que (SQL)

Limite de SGBD Relationnel

Faiblesse du Modèle Relationnel

- Modèle de données simple : difficile de modéliser des structures complexes :
  - ✓ pas d'attribut complexe, ni multivalué
  - ✓ Données alphanumériques uniquement (pas d'images, de sons, de vidéo, …)
  - ✓ un seul type de lien (clé étrangère) : jointure indispensable
  - ✓ le système de types figé (non extensible)
  - ✓ Les objets sont éclatés en plusieurs relations, ce qui
  - ✓ multiplie les jointures dans les requêtes des utilisateurs

Limite de SGBD Relationnel

Faiblesse du Modèle Relationnel

- Le développement d'applications est très lent et les applications sont difficilement maintenables.
- Mal adapté à :
  - ✓ de nouvelles applications : aide à la décision, conception, géographie, bureautique, multimédia, …
  - ✓ de nouvelles techniques : IHM, programmation orientée objet, programmation en logique, architectures réparties,…

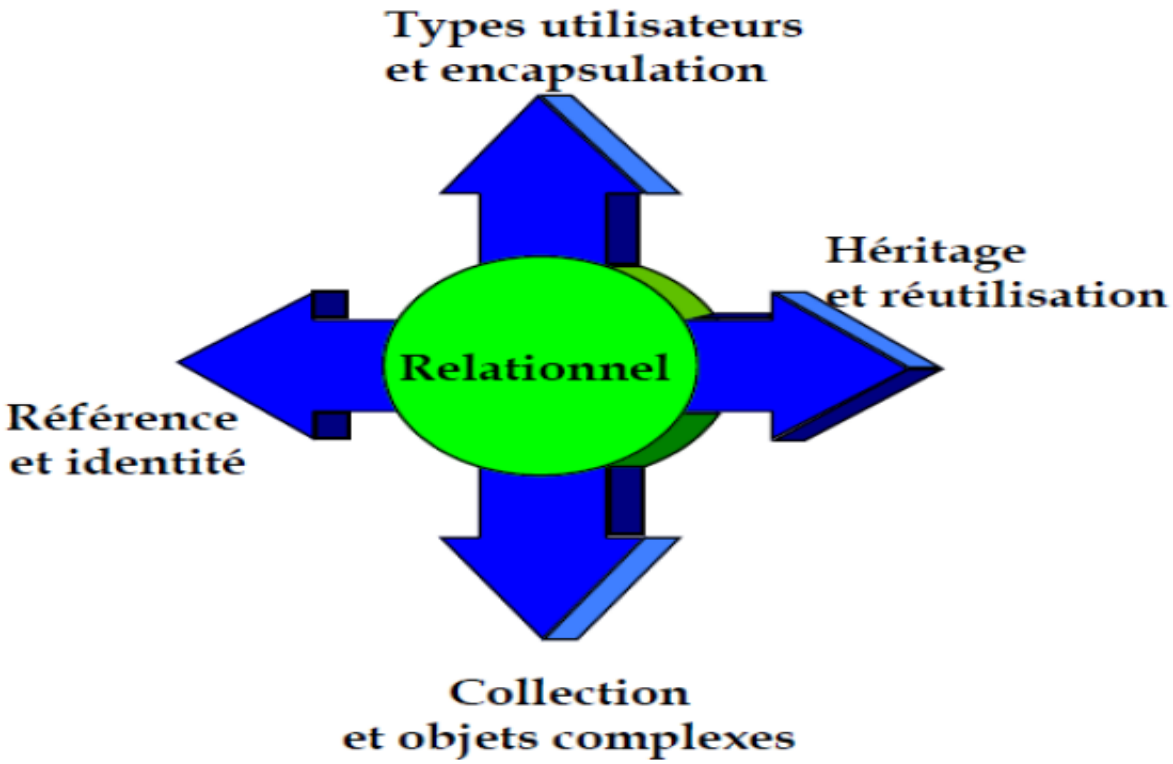
Nouveau Besoin

- Les nouvelles applications ont besoin de gérer des objets complexes comme : texte, graphiques, cartes, images, … Pour cela, les bases de données objets sont nées, ils regroupent:
  - les bases de données
  - et les langages de programmation orientés objets (Simula, Eiffel,Smalltalk, C++, JAVA). Ces langages permettent d"augmenter la productivité des développeurs, ils se basent sur:
    - Les objets (valeurs, méthodes)
    - L'héritage, permet de donner à une classe fille les même propriétés de la classe mère sans avoir à les redéfinir.
- Deux solutions ont été proposés afin de pallier les lacunes de modèle relationnel:

1) Modèle Orienté Objet.
2) **Modèle Objet-Relationnel.**

Les extensions apportées au relationnel

❖ Le modèle objet-relationnel est fondé sur l'idée d'extension du modèle relationnel avec les concepts essentiels de l'objet.



❖ Le cœur du modèle reste donc conforme au relationnel, mais l'on ajoute les concepts clés de l'objet sous une forme particulière pour faciliter l'intégration des deux modèles.

Les extensions apportées au relationnel

- Le modèle **objet-relationnel** conserve :
  - Les notions de base du relationnel :domaine, table, attribut, clé et contrainte référentielle
  - Les concepts **de l' objet:** de **type utilisateur** avec structure éventuellement privée et opérations encapsulant, de collections supportant des attributs multivalués (structure, liste, tableau, ensemble, etc.), **d'héritage** sur relations et types, et d'identifiants d'objets permettant les références intertables. Les domaines peuvent dorénavant être des types utilisateurs.

Les extensions apportées au relationnel

Deux vues sont possibles :

- la vue **relationnelle** dans laquelle les relations sont des relations entre objets chaque colonne représente un objet ;
- la vue **objet** où une ligne d'une table peut correspondre à un objet simple (un tuple) ou complexe (un tuple avec des attributs complexes).

- *Exemple:*

Objet conducteur

Objet police

Police	Nom	Adresse	Conducteurs		Accidents		
			Conducteur	Âge	Accident	Rapport	Photo
24	Paul	Paris	Paul	45	134		
			Robert	17	219		
					037		

Objet Police

Les relations dans les BDOR ne suivent pas la 1 ère forme normale

Les extensions apportées au relationnel

Les concepts additionnels essentiels

Les extensions du langage SQL

Au-delà du modèle OR, il est nécessaire d'étendre le langage de manipulation de données pour supporter les nouvelles avancées du langage de définition.

- SQL 3 (SQL 99) étend le SQL aux concepts Objet.

Principe

- Typage fort:

Type = Données + Méthodes

Les Fonctionnalités Objet De SQL3 :

- ❖ Définition De Types Abstraits,
- ❖ Support D'objets Complexes,
- ❖ Héritage De Type Et De Table.

1 Types définie par l'utilisateur (UDT)

- User data type (UDT)
- Le développeur peut créer ses propres types de données.
- Équivalent à une classe en OO
- Un UDT possède
  - Des attributs (simples ou autres types)
  - Des méthodes
- Utilisations d'un UDT
  - Comme colonne d'une table ordinaire,
  - Comme attribut d'un autre type,
  - Comme une ligne d'une table d'objets

1 Types définie par l'utilisateur (UDT)

1) Types distinct

- Le mot clé **DISTINCT** est utilisé pour renommer un type de base existant déjà
- Permet de différencier les domaines des colonnes
- Les types DISTINCT Sont définis à partir des types de base et s'utilisent avec les mêmes instructions que ces types de base
- Exemple

```
CREATE DISTINCT TYPE matricule AS INTEGER
```

1 Types définie par l'utilisateur (UDT)

2) Types Varray ou Table

- **VARRAY<longueur>** : créer des attributs multivalués sous la forme de tableaux à une dimension limitée.
- **TABLE :** créer des attributs multivalués sous la forme de tableaux à une dimension non limite.
- Syntaxe:

```
CREATE  TYPE <nom-type>  AS (VARRAY <longueur> | TABLE)
OF <nom-type2>;
```

Exemple :

```
Create TYPE Telephone as Table of varchar(10);
```

```
CREATE TYPE TPrénoms AS VARRAY(4) OF VARCHAR(20)
```

Exemple de valeur : `('Fatima', 'Amina')`

```
CREATE TABLE personne
  ( NSS INTEGER , nom VARCHAR(20) , prénoms TPrénoms )
```

```
INSERT INTO personne (NSS, nom, prénoms)
VALUES ( 1548401, 'Brahimi',
         TPrénoms ('Fatima', 'Amina') )
```

1 Types définie par l'utilisateur (UDT)

3) Types structure

- Correspondent aux classes des langages objets.
- Ils peuvent contenir des constructeurs, attributs(≈ variables d'instances), fonctions et des procédures (≈ méthodes)
- Les membres peuvent être public, protected ou private.
- Les fonctions et les procédures peuvent être écrites en SQL ou en un autre langage.
- Supportent l'héritage

Création d'un type de données

- ✓ La syntaxe est semblable à celle de la création d'une table:
- ✓ Un type ne peut contenir des contrainte d'intégrité.

```
create type nom-type  as object (
        attribut1   type  ,
        attribut2   type ,
         attribut3    type
 )
```

- ✓ La commande « **CREATE OR REPLACE TYPE »** permet de redéfinir in type s'il existe déjà.
- ✓ Type peut être :

1) **Un type usuel de SQL** (CHAR, VARCHAR, NUMBER…)

2) **Un type défini par l'utilisateur**

Exemple

```
create type departement as object
(
        numdept integer ,
        nomdept varchar (20) ,
        lieu varchar(20)
  )
```

```
CREATE TYPE Tadresse AS OBJECT
(  rue VARCHAR(20),
   numéro VARCHAR(4),
   ville VARCHAR(20)
)
```

Création des Méthode

✓ Un type OBJECT peut avoir des méthodes

```
CREATE TYPE nom-type AS OBJECT
      (    <déclaration des attributs> ,
           <déclaration des signatures des méthodes>
       )
```

✓ Le corps contient le code de méthodes

- Il peut contenir
  - ❖ Des instructions SQL ou d'un langage de programmation (JAVA)
  - ❖ Appels de méthodes

Exemple

```
CREATE TYPE departementType AS OBJECT
  (
      numDept integer,
      nomDept VARCHAR(30),
      lieu  varchar(30),
      MEMBER FUNCTION getLieu RETURN VARCHAR
 )
```

```
CREATE TYPE BODY departementType AS
   MEMBER FUNCTION getLieu RETURN varchar IS
      begin
            return lieu;
            end;
      end;
```

L'héritage

- Les types supportent l'héritage multiple avec le mot-clé UNDER
- Héritage des attributs et méthodes
- Possibilité de redéfinir le code des méthodes dans les sous-types

```
CREATE TYPE TPersonne AS OBJECT
  ( NSS INTEGER , nom VARCHAR(20) , prénom
    VARCHAR(20), conjoint REF TPersonne  )
    NOT FINAL

CREATE TYPE TEtudiant UNDER TPersonne
  (  faculté VARCHAR(18), cycle VARCHAR(18)  )
```

- Un type est final par défaut
- Le mot clé NOT FINAL est obligatoire si le type a des sous-types

Modification d'un type de données

- L'ajout d'un attribut dans un type

```
Alter type <nom_type> add attribute <nom_attribute>
<type_attribute> cascade ;
```

Alter type <nom_type> add attribute <nom_attribute>
<type_attribute> cascade ;

Exemple :

```
Alter type Employe_Type add attribute
date_naissance date
cascade;
```

Cascade : propage aux tables construites à partir du type.

- L'ajout dune méthode dans un type

```
Alter type <nom_type> add member Function
<nom_fonction> [(<nom_parametre>, …)]
return
<type_de retour> ;
```

Exemple :

```
Alter type Employe_Type add member function age
return integer cascade;
```

Supprimer un type de données

- Il est possible de supprimer des types déjà crées en utilisant la clause

```
drop type nom_Type ;
```

- Il faut d'abord supprimer les tables construites à partir de ce type avec (drop table) , ainsi que les type construits

Exemple :

```
drop table personne;
drop type type_personne;
drop type adresse;
```

4) Types de ligne

- SQL99 possède aussi la notion de type de ligne qui correspond aux structures du C: c'est un ensemble non encapsulé d'attributs.
- On peut ensuit utiliser ce type pour une déclaration d'attribut ou même pour créer une table à partir de ce type (comme pour les autres types)

1) Type de ligne non nommé

```
Create table Employer (
nomE varchar (35),
Adresse ROW(numero integer , rue varchar(30), ..)
)
```

2) Type de ligne nommé

```
Create ROW type adresse_t (
   numero integer ,
    rue varchar(30), ..
  )
```

Création des Table a partir des type

La création de type ne crée pas d ’objets

- Les objets d’un type sont persistants que lors ils sont insérés dans des tables déclarées
- Il est possible de créer trois types de tables :
  - ✓ Tables du relationnel classique
  - ✓ **Tables de valeurs structurées**
  - ✓ **Tables à partir d’un type de données**

(1) Table de valeur structurer

```
CREATE TABLE personne
   ( NSS INTEGER , nom VARCHAR(20) , prénoms
     TPrénoms, adr Tadresse )
```

```
CREATE TYPE TPrénoms AS VARRAY(4) OF
      VARCHAR(20)
```

```
CREATE TYPE Tadresse AS OBJECT
(   rue VARCHAR(20), numéro VARCHAR(4), ville
    VARCHAR(20)   )
```

(2) Des tables à partir des types

```
CREATE TYPE TEmploye AS OBJECT
  (  matricule INTEGER , nom VARCHAR(20) ,
     prénoms TPrénoms, adr Tadresse,
     dept INTEGER, salaire INTEGER     )

CREATE TABLE employe OF TEmploye
```

- On peut aussi indiquer des contraintes d'intégrité

```
CREATE TABLE employe OF TEmploye
      (  primary key (matricule) )
```

Caractéristique d'une table objet –relationnelle

- Une table peut hériter d'une ou plusieurs tables. (Pas supporté par Oracle10g et 11g)

```
Create Table sportif under personne ( niveau varchar(10))
```

- Les lignes des tables objet-relationnelles sont considérées comme des objets avec des identifiants (OID)
- On peut utiliser des références pour désigner les lignes de ces tables

Requête

1) Insertion dans une table

- On ajout des données comme avec une table relationnelle normale:

```
INSERT INTO employe (matricule , nom,… )
VALUES (5423, 'Ben Ali', …)
```

- On peut aussi utiliser le « **constructeur du type** » avec lequel la table a été construite:

```
INSERT INTO employe values (  TEmplye (5423,
'Ben Ali', …) )
```

- Si le type est un type utilisé par un autre type, l'utilisation du constructeur du type est obligatoire:

```
INSERT INTO employe (matr, nom, sal, adresse)
Values (1,'Toto', 12000, adresse_type(12,'Victor
Hugo ', 'Nice'))
```

2) afficher les donner d'une table

```
Select nom, e.adresse.rue
from employe e
```

2) Modification

- On utilise la notation pointée (.) pour accéder aux attributs

```
UPDATE employe SET employe.salaire = 12000
WHERE employe.nom = 'Ben ali'
```

```
UPDATE employe e
SET salaire =12000, e.adresse.numero=23
WHERE nom= 'Dupond';
```

- On utilise la double notation pointée (..) pour accéder aux attributs d'une colonne de type structuré

```
UPDATE employe
SET employe.adresse..numero = 12
WHERE employe.nom = 'Ben ali'
```

2) Apelle de procédure ou fonction

```
SELECT e.NOM, e..AGE()
FROM employe e
WHERE e..AGE() < 35;
```

Références

- SQL3 permet de parcourir les associations représentées par des attributs de type références
- Dans la définition d'un type, on peut indiquer qu'un attribut contient des références (et non des valeurs) à des données d'un autre type
- Si l 'objet est de type T, sa référence est de type REF(T)
- Syntaxe: REF nom-du-type

Exemple :

```
CREATE TYPE Employer_type as object (
matricule integer,
nom VARCHAR (30),
…
dept REF dept_type  ) ;
```

1) Requêtes avec références - Sélection

La notation pointé permet de récupérer les attributs d'un type dont on a un pointeur

- Lieu de travail des employer (avec oracle) :

```
SELECT nom , e.dept.lieu
FROM employer e
```

- En SQL 99 :

```
SELECT nom , e.dept -> lieu
FROM employer e
```

2) Requêtes avec références - Insertion

- Requête avec pointeur null

```
INSERT INTO employer VALUES (9123, 'Ahmed',..,
NULL)
```

- Requête avec une référence vers le département numéro 10

```
INSERT INTO employer (matricule, nom, dept)
VALUES (  9123, 'Ahmed',
          SELECT REF(d) FROM dept d
          WHERE dept.numdept = 10 )
```

3) Requêtes avec références - Modification

```
UPDATE employer
SET dept = ( SELECT REF(d)
                    FROM dept d
                    WHERE numDept = 10  )
WHERE matricule = 9123 ;
```

Collections

- Permettent de représenter des colonnes multivaluées
- Types de collections:
  - ✓Ensemble (sans doublons)
  - ✓Sac (avec des doublons)
  - ✓Liste (ordonnée et indexée par un entier)
  - ✓D'autres types de collections peuvent être ajoutées par les SGBD
- Les constructeurs de collections peuvent être appliqués sur tout type déjà défini39

Exemple :

```
CREATE TYPE TEmploye AS OBJECT
      (  matricule INTEGER, nom VARCHAR(30),
         prenoms LIST ( VARCHAR (15)),
         enfants SET  ( personne ),   …)

CREATE TABLE employe OF TEmploye
```

- Les collections peuvent être utilisées comme des tables en utilisant le mot-clé TABLE

```
SELECT nom FROM employe e
WHERE nom IN (  SELECT *
                FROM TABLE ( e.prenoms ) )
```

_ Utilisation des collection

- Ajout d'une colonne à une table

  ✓Nous allons ajouter la colonne passetemps (ensemble de chaînes de caractères) à la table personne

```
ALTER TABLE personne ADD COLUMN passetemps
SET (VARCHAR)
```

- Donner la requête permettant de retrouver les références des personnes ayant pour passe-temps le vélo

- Réponse:

```
SELECT REF(p) FROM personne p
WHERE 'VELO' IN
              SELECT * FROM TABLE (p.passetemps)
```

2) Types Varray ou Table

- **VARRAY<longueur>** : créer des attributs multivalués sous la forme de tableaux à une dimension limitée.
- **TABLE :** créer des attributs multivalués sous la forme de tableaux à une dimension non limite.
- Syntaxe:

```
CREATE  TYPE <nom-type>  AS (VARRAY <longueur> | TABLE)
OF <nom-type2>;
```

Exemple :

```
CREATE TYPE TPrénoms AS VARRAY(4) OF VARCHAR(20)
```

Exemple de valeur : `('Fatima', 'Amina')`

```
CREATE TABLE personne
  ( NSS INTEGER , nom VARCHAR(20) , prénoms TPrénoms )
```

```
INSERT INTO personne (NSS, nom, prénoms)
VALUES ( 1548401, 'Brahimi',
         TPrénoms ('Fatima', 'Amina') )
```

❖ Oracle 11g n‘‘offre que 2 types de collections:

- Table imbriquée (NESTED TABLE) qui est une collection non ordonnée et non limitée en nombre d’éléments.
- Tableau prédimensionné (VARRAY) qui est une collection délément de même type, ordonnée et limitée en taille.

Stockage des Collections libre (table imbriquées)

- Construction dune **NESTED TABLE:**

1. Création du type de la collection, avec:

```
CREATE  TYPE <nom-type>  AS  TABLE of Type-elt  ;
```

2. Création du type contenant la collection puis 3, ou création directement de la table, par exemple personne.

3. Création de la table de ce dernier type (sil y a lieu).

- Génération automatique d'un identifient superficiel NESTED_TABLE_ID pour chaque collection
- Possibilité de créer un index dans la NESTEDTABLE